

A Aufgabenstellung `noten.py`

Lehrer müssen oft Noten- und Schülerlisten in verschiedenen Formaten zusammenführen. Entwickle ein kleines Hilfsprogramm `noten.py` für die Kommandozeile (CLI), die einem Lehrer beim Zusammenführen einer Noten- und Schülerliste unterstützt.

Wir verwenden dazu *Regular Expressions*¹ zum Einlesen einer `.xml`-Datei für die Stammdaten der Schüler und üben wieder den Umgang mit `argparse`. Die mächtige Bibliothek `pandas`² hilft uns dabei,

- die `.csv`-Datei mit den Schülernoten einzulesen,
- die Notenliste und die Schülerliste zu verknüpfen,
- das Ergebnis zu filtern
- und wieder in eine `csv`-Datei zu schreiben.

B Hinweise:

B.1 Allgemeine Hinweise:

- Benutze die Python-Bibliothek `argparse`
- Überprüfe immer vor dem Einlesen einer Datei, ob diese auch existiert³
- Fehlermeldungen sollen – wie unter Linux üblich – auf dem `Stderror`-Device ausgegeben werden⁴. Benutze standardisierte Fehlermeldungen⁵
- Benutze die beiliegenden Dateien, um die Lösung zu überprüfen
- Alle Dateien enthalten den Namen des Autors / der Autorin
- Der Code ist lesbar und gemäß PEP 8 (Style Guide for Python Code) formatiert: Einrückung, etc.
- Dokumentation mit PyDoc bei jeder Klasse / Methode / Funktion; alle Variablen und Methoden müssen sinnvolle Namen haben
- Type-Annotations = Type-Hints bei Argumenten von Methoden und Return-Values verwenden⁶
- “Hauptprogramm” mit `if __name__ == "__main__":`

B.2 Hinweise zu Git:

- Alle Beispiele kommen in das Github Repository, Dateiname: `noten.py`.
- **Achtung:** der `venv` Ordner kommt **nicht** in das Git-Repository – sondern wird in `.gitignore` eintragen
- Ab sofort wird bei jedem Python-Projekt die Datei `requirements.txt` in das Repository aufgenommen. Diese Datei beinhaltet eine Liste der benötigten Pakete. Man erzeugt sie mit:

```
pip freeze > requirements.txt
```

- Diese Bibliotheken können dann sehr einfach wieder installiert werden:

```
pip install -r requirements.txt
```

C Beispielaufufe

Die folgenden Beispielaufufe führen zu den beigefügten Dateien.

C.1 Hilfe

```
python noten.py -h
```

¹`import re`

²https://pandas.pydata.org/docs/user_guide/

³<https://pythonspot.com/read-file/>

⁴<https://www.w3docs.com/snippets/python/how-do-i-print-to-stderr-in-python.html>

⁵<https://www.geeksforgeeks.org/python-os-stderr-method/>

⁶siehe z.B. https://mypy.readthedocs.io/en/stable/cheat_sheet_py3.html

```
usage: noten.py [-h] [-n N] [-s S] [-m M] [-f F] [-v | -q] outfile
```

noten.py by Max Mustermann / HTL Rennweg

positional arguments:

outfile Ausgabedatei (z.B. result.csv)

options:

-h, --help show this help message and exit
-n N csv-Datei mit den Noten
-s S xml-Datei mit den Schülerdaten
-m M Name der Spalte, die zu verknüpfen ist (default = Nummer)
-f F Name des zu filternden Gegenstandes (z.B. SEW)
-v, --verbose Gibt die Daten Kommandozeile aus
-q, --quiet keine Textausgabe

C.2 Ohne Filter, verbose

Verknüpft beide Dateien. Man erhält eine .csv-Datei mit den relevanten Daten aller SchülerInnen und sämtlichen Gegenständen. Das Programm gibt über stdout den folgenden Text aus.

```
python noten.py -n noten.csv -s schueler.xml -v result_all.csv
```

csv-Datei mit den Noten : noten.csv

xml-Datei mit den Schülerdaten : schueler.xml

Name der Spalte, die zu verknüpfen ist : Nummer

Output-Datei: result_all.csv

C.3 Mit Filter auf SEW und ITP, weder verbose noch quiet

Hier erhält man als Ergebnis nur die Noten aus den angegebenen Gegenstand.

Bonus: Man kann mehrere Gegenstände angeben; das Programm bildet eine zusätzliche Spalte *Schnitt*, um die Durchschnittsnote der ausgewählten Gegenstände zu speichern.

```
python noten.py -n noten.csv -s schueler.xml -f SEW,ITP result_SEW.csv
```

Output-Datei: result_SEW.csv

D Hinweise und Tipps:

- Die .xml-Datei soll nicht mittels Pandas-Methode eingelesen werden, sondern mit der zu implementierenden Methode `read_xml(filename: str) -> pd.DataFrame`. Lies die gesamte Datei ein – z.B. mit `content = f.read()`. Finde mittels Regular Expression⁷ alle Treffer für *Nummer*, *Anrede*, *Vorname*, *Nachname*, *Geburtsdatum* und speichere sie in einem Pandas-DataFrame.

```
pattern = re.compile("regular expression", flags=re.DOTALL)
```

```
result = re.findall(pattern, content)
```

```
df = pd.DataFrame(result, columns=[.....], dtype=str)
```

- Die .csv-Dateien kann man mittels Hilfsfunktion manuell parsen – oder eine geeignete Pandas-Methode zum Lesen/Schreiben verwenden.
- Zum Verknüpfen der DataFrames eignet sich die Methode `pd.join()`. Tipp: setze den Index der DataFrames auf die geeignete Spalte.

⁷https://www.w3schools.com/python/python_regex.asp